Лабораторна робота № 3

Мета роботи:

Сформувати практичні навички створення графічного інтерфейсу користувача (GUI) у ROS2, навчитися інтегрувати GUI з нодами ROS2, реалізувати передачу команд руху через Topic /cmd_vel, а також організувати відображення статусу системи.

Репозиторій з роботою: https://github.com/zakappaa/ROS2_Ginzburg

Хід роботи

Підготовка середовища

Роботу виконано в тій самій віртуальній машині та робочому просторі, що й попередні лабораторні роботи. Працездатність команд ros2 node list і ros2 topic list перевірено (рис. 1).

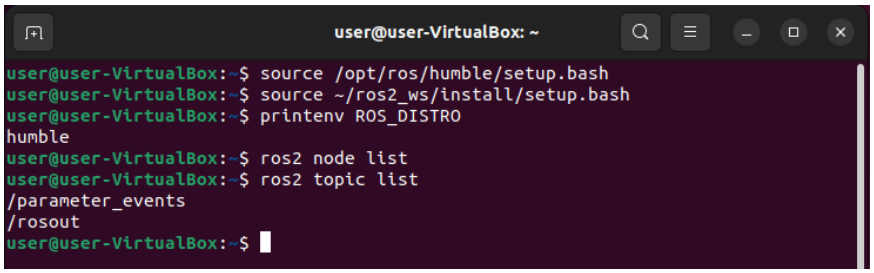


Рисунок 1 - Активація середовища та перевірка команд ROS2

Запущено симулятор turtlesim і motion node з Лабораторної роботи №2. Обидві ноди відображаються у виводі (рис. 2).

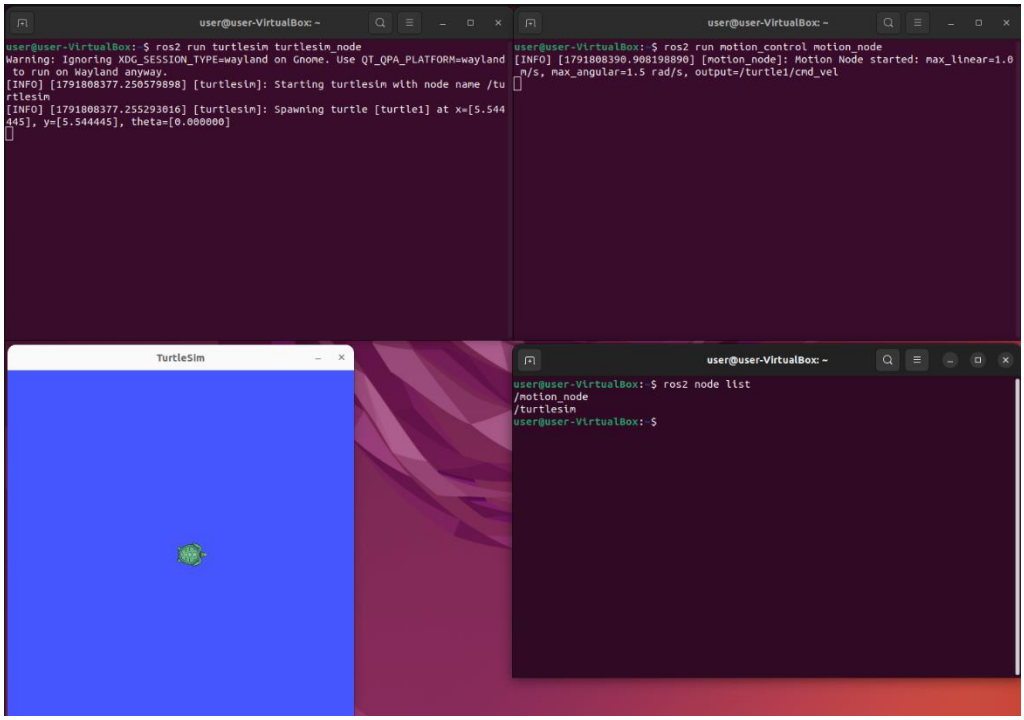


Рисунок 2 - Запуск turtlesim та motion nhode з Лабораторної роботи №2

Створення GUI

У ~/ros2_ws/src створено пакет robot_gui командою:

ros2 pkg create --build-type ament_python --license Apache-2.0 --dependencies rclpy geometry_msgs std_msgs --node-name gui_node robot_gui

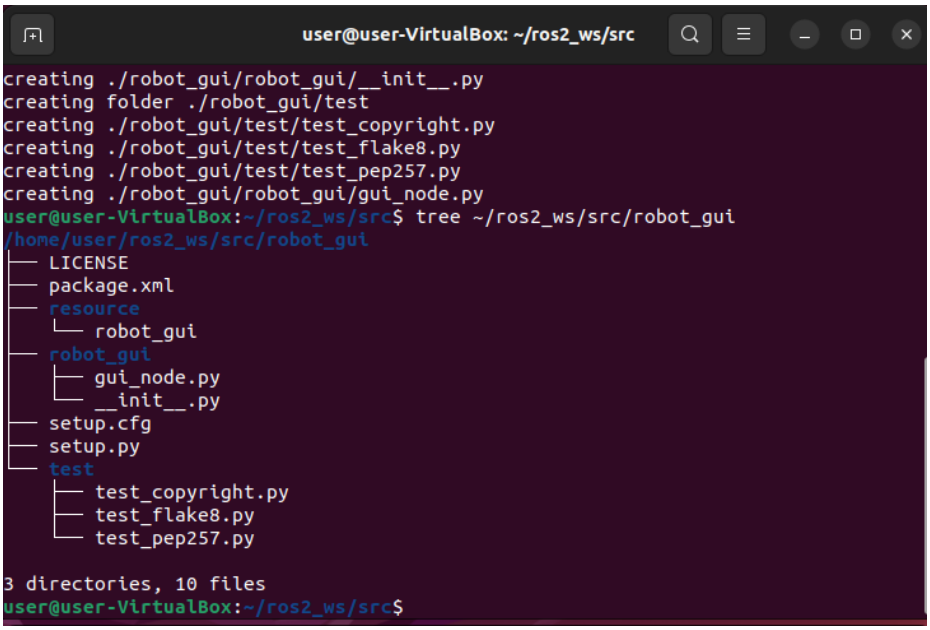


Рисунок 3 - Створення пакету robot_gui та його структура

Пакет зібрано командою colcon build --packages-select robot_gui. Коректність встановлення перевірено командами ros2 pkg list і ros2 pkg executables robot_gui (рис. 4).

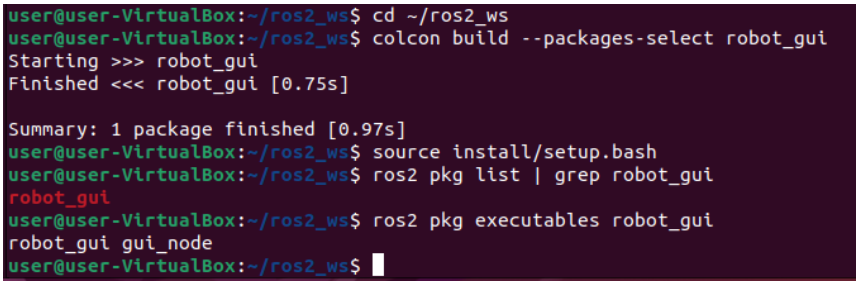


Рисунок 4 - Збірка та перевірка встановлення пакету robot_gui

Реалізація GUI-ноди

GUI-нода реалізована за допомогою двох класів:

1. **RobotGuiNode** публікує команди Twist у топік /cmd_vel, підписана на статус системи /motion_status (String) та містить метод send_command для відправки швидкостей.
2. **ControlWindow** - вікно на базі PyQt5, що містить:
   - поля налаштування лінійної та кутової швидкостей (QDoubleSpinBox)
   - 5 кнопок руху («Вперед», «Назад», «Ліворуч», «Праворуч», «Стоп»)
   - текстове поле для виводу повідомлень з /motion_status та мітку останньої надісланої команди.

При натисканні кнопки метод on_button формує вектор швидкості (linear.x, angular.z) відповідно до заданих параметрів і передає його ноді. Розділення коду на два класи ізолює логіку зв'язку ROS2 від графічного інтерфейсу.

Таблиця 1 - Відповідність кнопок GUI командам руху

Кнопка	linear.x	angular.z	Рух робота
Вперед	+v	0	рух вперед
Назад	−v	0	рух назад
Ліворуч	0	+w	поворот ліворуч на місці
Праворуч	0	−w	поворот праворуч на місці

Кнопка	linear.x	angular.z	Рух робота
Стоп	0	0	зупинка

де v і w - значення з полів «Лінійна швидкість» і «Кутова швидкість».

Фрагмент коду створення GUI:

```
class ControlWindow(QWidget):


    def __init__(self):
        super().__init__()
        self.node = RobotGuiNode(self.show_status)
        self.setWindowTitle('Керування роботом (ROS2)')
        self.setMinimumWidth(420)

        self.linear_speed = self.make_spinbox(0.5)
        self.angular_speed = self.make_spinbox(1.0)
        speeds = QFormLayout()
        speeds.addRow('Лінійна швидкість, м/с:', self.linear_speed)
        speeds.addRow('Кутова швидкість, рад/с:', self.angular_speed)

        buttons = QGridLayout()
        self.add_button(buttons, 'Вперед', 0, 1, 1, 0)
        self.add_button(buttons, 'Ліворуч', 1, 0, 0, 1)
        self.add_button(buttons, 'Стоп', 1, 1, 0, 0)
        self.add_button(buttons, 'Праворуч', 1, 2, 0, -1)
        self.add_button(buttons, 'Назад', 2, 1, -1, 0)

        self.command_label = QLabel('Остання команда: —')
        self.status_box = QPlainTextEdit()
        self.status_box.setReadOnly(True)
        self.status_box.setMaximumBlockCount(200)

        layout = QVBoxLayout(self)
        layout.addLayout(speeds)
        layout.addLayout(buttons)
        layout.addWidget(self.command_label)
        layout.addWidget(QLabel('Статус системи (/motion_status):'))
        layout.addWidget(self.status_box)
```

```
        self.ros_timer = QTimer(self)
        self.ros_timer.timeout.connect(
            lambda: rclpy.spin_once(self.node, timeout_sec=0))
        self.ros_timer.start(50)

    def add_button(self, grid, text, row, col, linear_dir, angular_dir):
        button = QPushButton(text)
        button.setMinimumSize(110, 50)
        button.setAutoRepeat(bool(linear_dir or angular_dir))
        button.clicked.connect(
            lambda: self.on_button(text, linear_dir, angular_dir))
        grid.addWidget(button, row, col)
```

Фрагмент коду Publisher і підписки на статус:

```
class RobotGuiNode(Node):

    def __init__(self, on_status):
        super().__init__('robot_gui')
        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)
        self.subscription = self.create_subscription(
            String, '/motion_status', lambda msg: on_status(msg.data), 10)

    def send_command(self, linear_x, angular_z):
        msg = Twist()
        msg.linear.x = linear_x
        msg.angular.z = angular_z
        self.publisher_.publish(msg)
        self.get_logger().info(
            f'Published /cmd_vel: linear.x={linear_x:.2f}, angular.z={angular_z:.2f}')
```

Для відображення статусу motion node з Лабораторної роботи №2 доповнено Publisher у топік /motion_status. Після обробки кожної команди нода публікує рядок статусу: OK (команду передано без змін) або LIMITED (швидкість обмежено) разом з фактично надісланими роботу значеннями:

```
                limited = (cmd.linear.x, cmd.angular.z) != (msg.linear.x, msg.angular.z)
            status = String()
            status.data = (f'{"LIMITED" if limited else "OK"}: '
                           f'linear.x={cmd.linear.x:.2f}, angular.z={cmd.angular.z:.2f}')
            self.status_publisher.publish(status)
```

Тестування

GUI-ноду запущено командою. Відкрилося вікно керування з полями швидкості, п'ятьма кнопками та полем статусу (рис. 5). До запуску motion node натискання кнопок оновлювало рядок «Остання команда», але черепаха не рухалася і статус не надходив. Це підтверджує, що GUI не керує роботом напряму, а лише публікує команди в /cmd_vel.

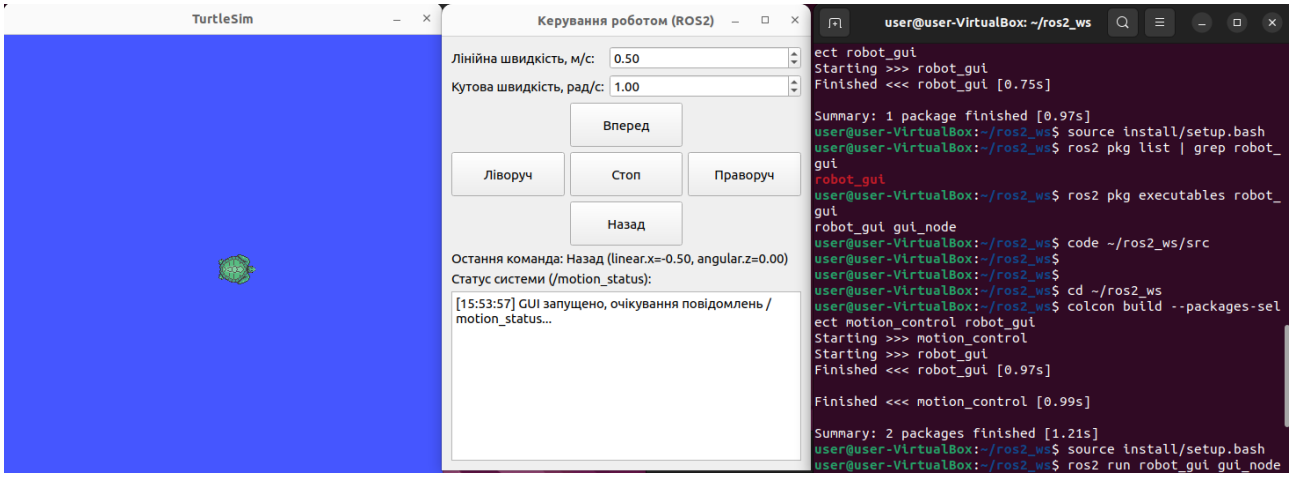


Рисунок 5 - Графічний інтерфейс керування роботом

Після запуску ros2 run motion_control motion_node робот у симуляторі почав виконувати команди кнопок. У полі статусу GUI відображалися повідомлення від motion node з фактичними швидкостями (рис. 6).

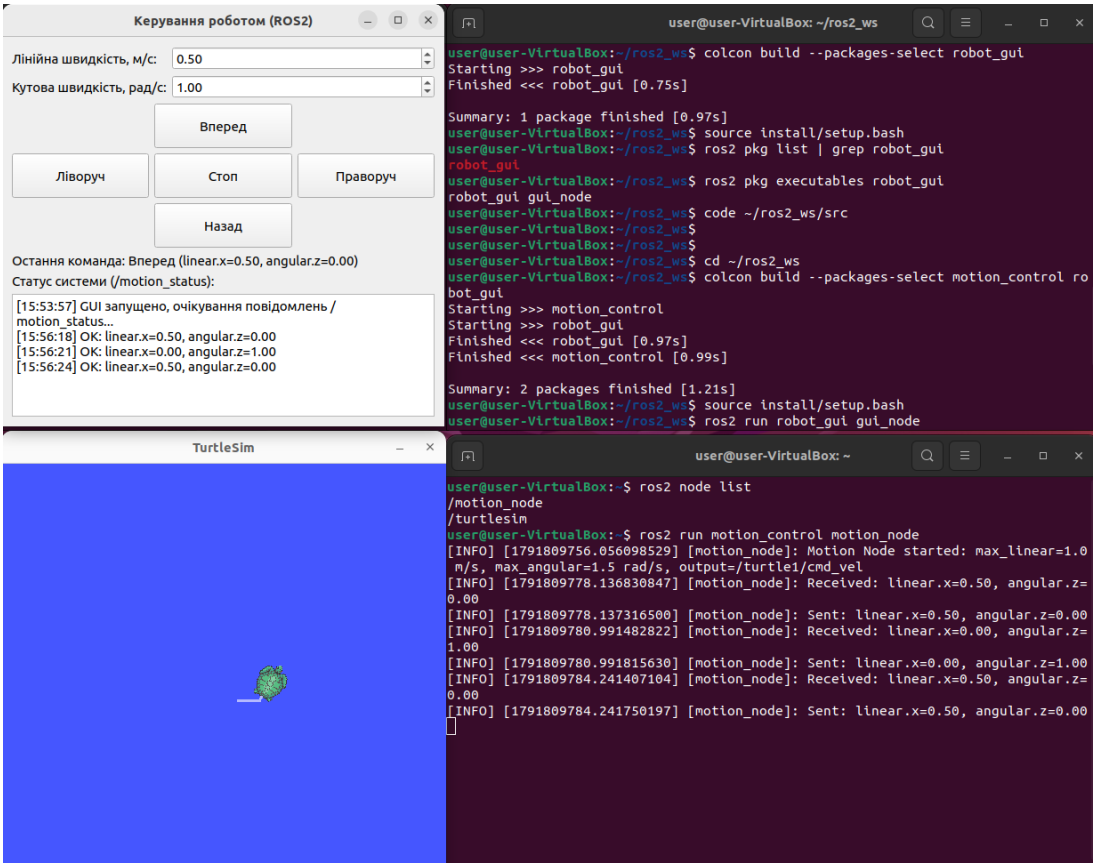


Рисунок 6 - Керування роботом за допомогою кнопок GUI

Коректність передачі команд перевірено командами ros2 topic echo /cmd_vel і ros2 topic echo /motion_status. Значення полів linear.x і angular.z для кожної кнопки відповідають Таблиці 1 (рис. 7).

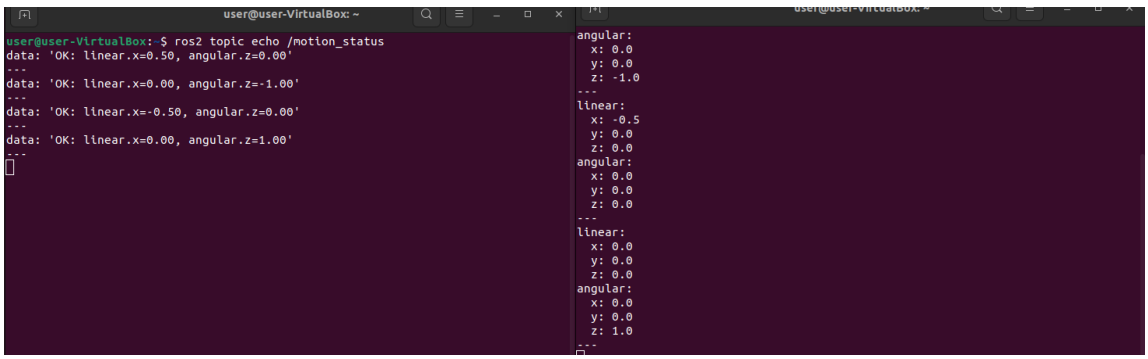
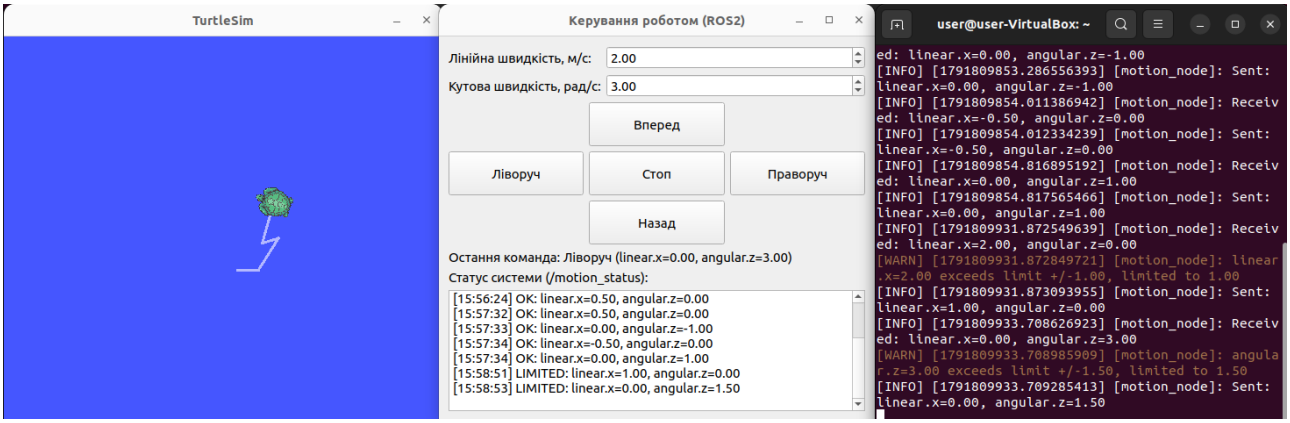


Рисунок 7 - Перевірка передачі команд через echo

При встановленні в GUI швидкостей, що перевищують допустимі пороги motion node (2.0 м/с і 3.0 рад/с), motion node обмежив їх до 1.0 м/с і 1.5 рад/с. У полі статусу GUI з'явилися повідомлення LIMITED, у журналі Motion Node — WARN-повідомлення (рис. 8).



Нисунок 8 - Відображення в GUI статусу обмеження швидкості

Висновки

У ході лабораторної роботи розроблено GUI-ноду robot_gui на основі Qt, що керує роботом через стандартний топік /cmd_vel. Інтерфейс містить кнопки «Вперед», «Назад», «Ліворуч», «Праворуч», «Стоп», поля для задання швидкостей і текстове поле статусу. Натискання кнопок формує повідомлення, які обробляє motion node з Лабораторної роботи №2. Motion node доповнено публікацією статусу в /motion_status, який GUI відображає користувачу. Інтеграцію циклів подій Qt і ROS2 реалізовано через таймер QTimer з викликом rclpy.spin_once. Коректність передачі команд підтверджено командами ros2 topic echo.